

History of DevOps

Traditional software development and IT operations teams had difficulties, which led to the creation of DevOps. These groups functioned independently in the past, frequently resulting in misunderstandings and delays. Combining the terms ‘development’ with ‘operations’, DevOps first gained popularity in 2009. The Agile development principles, which emphasised iterative development and cooperation, led to the movement. The goal was to create a more cohesive and effective workflow by applying these concepts to operations.

The goal of enhancing cooperation between development and operations teams is where DevOps got its start. The Agile technique, which sought to improve development processes, inspired it. The emphasis shifted over time to encompass continuous delivery and automation, giving rise to the contemporary DevOps methodologies we use today.

How DevOps gained popularity

A higher need for speed: Companies have to swiftly offer updates and new features in the quick-paced world of technology. DevOps techniques assisted in meeting this need by optimising processes and cutting down on the time between development and deployment.

Improved cooperation: DevOps promoted improved communication and cooperation between the development and operations teams by dismantling communication silos. This resulted in more effective procedures and fewer misunderstandings.

Tools and automation: Continuous integration and continuous deployment (CI/CD), two essential DevOps techniques, are now easier to adopt because of the development of solid automation tools and technologies. This automation sped up the entire

procedure by eliminating errors and manual labour.

Success stories: Several well-known businesses, including Netflix and Amazon, have implemented DevOps and shared their success tales. These illustrations of the concrete advantages of DevOps inspired other companies to adopt the same practices.

Advantages of DevOps

Faster time to market: DevOps concepts enable more rapid development cycles, which allows organisations to roll out updates and new features more quickly.

Improved collaboration: Teams who work together more closely have better communication, which leads to fewer errors and more neatly laid out work.

Increased productivity: Automating repetitive tasks reduces human error and speeds up processes, which improves workflow effectiveness.

Better software: Continuous testing and integration of software allows for the early detection and resolution of issues, resulting in software of a higher calibre.

Increased customer happiness: Faster updates and improvements lead to better user experiences and increased customer happiness.

The drawbacks of DevOps

Cultural transformation necessary: Embracing DevOps frequently necessitates a big, time-consuming, and difficult cultural transformation inside a business.

Initial costs: Although putting DevOps practices and technologies into operation can be expensive upfront, the long-term advantages typically surpass these costs.

Complexity: Combining different tools and methods can complicate things, which calls for specific knowledge and instruction.

Security issues: To prevent vulnerabilities, security must be

regularly addressed with increasing automation and frequent deployments.

Resources

- **Where to learn?**
 - **Coursera:** Many universities offer free courses on Coursera, such as ‘Introduction to DevOps’ by the University of California, Davis.
 - **edX:** Check out free courses like ‘DevOps for Developers’ by Microsoft.
 - **Udemy:** There are free introductory courses on Udemy, such as ‘DevOps Basics for Beginners’.
- **Free documentation and guidelines**
 - **Docker documentation:** Docker provides comprehensive, free resources for learning containerisation.
 - **Kubernetes documentation:** The official Kubernetes site has detailed guides and tutorials.
 - **GitLab documentation:** GitLab offers extensive documentation on CI/CD and DevOps practices.
 - **Terraform documentation:** HashiCorp provides guides and tutorials for infrastructure as code.

 **Note:** The information provided here is for resource recommendation purposes only and does not constitute an endorsement or promotion of any specific tools or services. The content is subject to change and may be updated periodically.

Common DevOps frameworks and instruments

Jenkins: A popular open source automation server for continuous delivery and integration (CI/CD). A wide range of plugins are supported by Jenkins to interface with other programs.

Docker: An application development, shipping, and operating system for containers. Consistency